RoboPet - Economy and Progression

Earn, Keys, Crates and Loot Variety

Cursor task - fix the new-player dead end and build the progression loop

A brand-new account has 0 scrap and 0 keys, so it cannot craft, open crates, or win fights - a hard dead end. This adds reliable income from caring, a key shop, crates with weighted loot, and a varied item catalog. Build in order:

1. Part 1 - Starter pack for new accounts (breaks the 0 / 0 deadlock)
2. Part 2 - Earn scrap from care actions (feed / pet / charge)
3. Part 3 - Keys + Shop (spend scrap on keys)
4. Part 4 - Crates + weighted loot tables
5. Part 5 - Item catalog (variety: modules, consumables, cosmetics)

After each part: flutter pub get, hot restart, verify the Acceptance Test.

Why a new account is stuck

With 0 scrap and 0 keys there is no way in: you cannot buy a key, so you cannot open a crate, so you have no modules, and combat alone was the only source of scrap. The fix gives two always-available income paths - a one-time starter pack and scrap from everyday care - so progression never deadlocks.

The progression loop

6. New account grants a starter pack (scrap + 1 key + a starter core).
7. Care for the pet (feed / pet / charge) to earn scrap.
8. Spend scrap on keys in the Shop.
9. Open crates with keys.
10. Collect varied items (modules / consumables / cosmetics) and equip a loadout.
11. Fight and win for more scrap and loot, then reinvest.

Acceptance test

- [ ] A brand-new account receives the starter pack once (scrap + 1 key + a starter core).
- [ ] Feeding / petting / charging grants scrap, but only when the action actually helps a stat.
- [ ] Scrap can buy Basic and Rare keys in the Shop; insufficient scrap is blocked.
- [ ] Opening a crate consumes a key and grants one item based on rarity weights.
- [ ] Crates can drop modules (all 5 slots), consumables, and cosmetics.
- [ ] A new player can reach a working combat loadout without ever winning a fight first.
- [ ] Higher-tier keys and crates give better rarity odds.

Part 1 - Starter pack for new accounts

Goal: break the 0 / 0 deadlock. Grant a small pack once, guarded by a persisted flag.

```
// Run once for a brand-new account (guard with a persisted flag).
Future<void> grantStarterPack() async {
  final prefs = await SharedPreferences.getInstance();
  if (prefs.getBool('starter_granted') ?? false) return;
  final eco = ref.read(economyProvider.notifier);
  eco.addScrap(60);
  eco.addKeys(KeyTier.basic, 1);
  ref.read(inventoryProvider.notifier).add(catalogById('core_kinetic_basic'));
  await prefs.setBool('starter_granted', true);
}
```

Call it on first launch / account creation. The starter core means the player can field a basic loadout and win Level 1 immediately.

Part 2 - Earn scrap from care

Goal: a reliable income that never depends on combat.

```
enum CareAction { feed, pet, charge }

const kCareReward = <CareAction, int>{
  CareAction.feed: 6,
  CareAction.pet: 4,
  CareAction.charge: 8,
};

// Grant scrap only when the action actually helped (stat was not already full),
// so players cannot farm by spamming a maxed stat.
void onCareAction(CareAction action) {
  final changed = applyCare(action); // your care logic returns whether a stat rose
  if (changed) {
    ref.read(economyProvider.notifier).addScrap(kCareReward[action]!);
  }
}
```

Optional: a small daily login bonus (for example +20 scrap) for extra momentum.

Part 3 - Keys and the Shop

Goal: turn scrap into keys. Keep premium gems earned-only (epic keys cost gems from bosses / achievements).

```
enum KeyTier { basic, rare, epic }

const kKeyPriceScrap = <KeyTier, int>{
  KeyTier.basic: 40,
  KeyTier.rare: 120,
  // epic is bought with earned premium gems, not scrap
};
const kKeyPriceGems = <KeyTier, int>{
  KeyTier.epic: 5,
};

bool buyKeyWithScrap(KeyTier tier) {
  final price = kKeyPriceScrap[tier];
  if (price == null) return false;
  final eco = ref.read(economyProvider.notifier);
  if (!eco.spendScrap(price)) return false; // returns false if not enough scrap
  eco.addKeys(tier, 1);
```

```
  return true;
}
```

Part 4 - Crates and weighted loot

Goal: spend a key to open a crate and roll one item by rarity weights.

```
import 'dart:math';

// A crate of a tier consumes a key of the same tier and rolls one item.
const kCrateWeights = <KeyTier, Map<Rarity, int>>{
  KeyTier.basic: {Rarity.common: 75, Rarity.rare: 22, Rarity.epic: 3},
  KeyTier.rare: {Rarity.common: 35, Rarity.rare: 55, Rarity.epic: 9, Rarity.legendary:
1},
  KeyTier.epic: {Rarity.rare: 40, Rarity.epic: 50, Rarity.legendary: 10},
};

GameItem? openCrate(KeyTier tier) {
  final eco = ref.read(economyProvider.notifier);
  if (!eco.spendKey(tier)) return null; // no key of this tier
  final rarity = _rollRarity(kCrateWeights[tier]!);
  final pool = kCatalog.where((it) => it.rarity == rarity).toList();
  final item = pool[Random().nextInt(pool.length)];
  ref.read(inventoryProvider.notifier).add(item);
  return item;
}

Rarity _rollRarity(Map<Rarity, int> weights) {
  final total = weights.values.fold<int>(0, (a, b) => a + b);
  var roll = Random().nextInt(total);
  for (final e in weights.entries) {
    if (roll < e.value) return e.key;
    roll -= e.value;
  }
  return weights.keys.first;
}
```

Part 5 - Item catalog (variety, detailed modifiers)

Goal: lots of different drops across all 5 module slots, plus consumables and cosmetics. Each module carries detailed modifiers (flat points and percentages), like your existing Titanium Armor style, instead of a single power number.

```
enum ItemKind { module, consumable, cosmetic }
enum Rarity { common, rare, epic, legendary }

// Detailed modifiers, like your existing system (flat points and percentages).
enum StatMod { atkFlat, atkPct, defFlat, defPct, maxHpFlat, spdFlat, critPct,
cooldownPct, chargeRateFlat }

class Modifier {
  final StatMod stat;
  final double value; // flat = points, pct = percent (10 means +10%)
  const Modifier(this.stat, this.value);
}

enum ConsumableEffect { healHp, restoreEnergy, grantScrap, fillCharge }

class ConsumableSpec {
  final ConsumableEffect effect;
  final int amount;
```

```
  const ConsumableSpec(this.effect, this.amount);
}

class GameItem {
  final String id;
  final String name;
  final ItemKind kind;
  final Rarity rarity;
  final ModuleSlot? slot;           // modules
  final DamageType? element;        // cores
  final List<Modifier> modifiers;   // modules: detailed stat changes
  final ConsumableSpec? consumable; // consumables
  const GameItem({
    required this.id,
    required this.name,
    required this.kind,
    required this.rarity,
    this.slot,
    this.element,
    this.modifiers = const [],
    this.consumable,
  });
}

const kCatalog = <GameItem>[
  // Cores (set element + attack and charge)
  GameItem(id: 'core_kinetic_basic', name: 'Kinetic Core', kind: ItemKind.module,
rarity: Rarity.common, slot: ModuleSlot.core, element: DamageType.kinetic, modifiers:
[Modifier(StatMod.atkFlat, 4)]),
  GameItem(id: 'core_emp_basic', name: 'EMP Core', kind: ItemKind.module, rarity:
Rarity.common, slot: ModuleSlot.core, element: DamageType.emp, modifiers:
[Modifier(StatMod.atkFlat, 4)]),
  GameItem(id: 'core_fire_charged', name: 'Plasma Core', kind: ItemKind.module, rarity:
Rarity.rare, slot: ModuleSlot.core, element: DamageType.fire, modifiers:
[Modifier(StatMod.atkFlat, 7), Modifier(StatMod.chargeRateFlat, 3)]),
  GameItem(id: 'core_acid_charged', name: 'Corrosive Core', kind: ItemKind.module,
rarity: Rarity.rare, slot: ModuleSlot.core, element: DamageType.acid, modifiers:
[Modifier(StatMod.atkFlat, 7), Modifier(StatMod.chargeRateFlat, 3)]),
  GameItem(id: 'core_kinetic_prime', name: 'Prime Kinetic Core', kind: ItemKind.module,
rarity: Rarity.epic, slot: ModuleSlot.core, element: DamageType.kinetic, modifiers:
[Modifier(StatMod.atkFlat, 10), Modifier(StatMod.atkPct, 8),
Modifier(StatMod.chargeRateFlat, 5)]),
  GameItem(id: 'core_inferno_legend', name: 'Inferno Core', kind: ItemKind.module,
rarity: Rarity.legendary, slot: ModuleSlot.core, element: DamageType.fire, modifiers:
[Modifier(StatMod.atkFlat, 14), Modifier(StatMod.atkPct, 12),
Modifier(StatMod.chargeRateFlat, 8)]),
  // Armor (defense + HP)
  GameItem(id: 'armor_plate_basic', name: 'Plate Armor', kind: ItemKind.module, rarity:
Rarity.common, slot: ModuleSlot.armor, modifiers: [Modifier(StatMod.defFlat, 4)]),
  GameItem(id: 'armor_alloy', name: 'Alloy Armor', kind: ItemKind.module, rarity:
Rarity.rare, slot: ModuleSlot.armor, modifiers: [Modifier(StatMod.defPct, 8),
Modifier(StatMod.maxHpFlat, 25)]),
  GameItem(id: 'armor_reactive', name: 'Reactive Armor', kind: ItemKind.module, rarity:
Rarity.epic, slot: ModuleSlot.armor, modifiers: [Modifier(StatMod.defPct, 15),
Modifier(StatMod.maxHpFlat, 60)]),
  // Wheels (speed + a little crit)
  GameItem(id: 'wheels_tread_basic', name: 'Tread Wheels', kind: ItemKind.module,
rarity: Rarity.common, slot: ModuleSlot.wheels, modifiers: [Modifier(StatMod.spdFlat,
4)]),
  GameItem(id: 'wheels_hover', name: 'Hover Wheels', kind: ItemKind.module, rarity:
Rarity.rare, slot: ModuleSlot.wheels, modifiers: [Modifier(StatMod.spdFlat, 7),
Modifier(StatMod.critPct, 4)]),
```

```
  GameItem(id: 'wheels_warp', name: 'Warp Treads', kind: ItemKind.module, rarity:
Rarity.epic, slot: ModuleSlot.wheels, modifiers: [Modifier(StatMod.spdFlat, 12),
Modifier(StatMod.critPct, 8)]),
  // Sensor (crit, plus attack at epic)
  GameItem(id: 'sensor_eye_basic', name: 'Optic Sensor', kind: ItemKind.module, rarity:
Rarity.common, slot: ModuleSlot.sensor, modifiers: [Modifier(StatMod.critPct, 4)]),
  GameItem(id: 'sensor_radar', name: 'Radar Sensor', kind: ItemKind.module, rarity:
Rarity.rare, slot: ModuleSlot.sensor, modifiers: [Modifier(StatMod.critPct, 9)]),
  GameItem(id: 'sensor_oracle', name: 'Oracle Sensor', kind: ItemKind.module, rarity:
Rarity.epic, slot: ModuleSlot.sensor, modifiers: [Modifier(StatMod.critPct, 12),
Modifier(StatMod.atkPct, 5)]),
  // Utility (faster cooldowns; negative cooldownPct means faster)
  GameItem(id: 'utility_coolant_basic', name: 'Coolant Unit', kind: ItemKind.module,
rarity: Rarity.common, slot: ModuleSlot.utility, modifiers:
[Modifier(StatMod.cooldownPct, -8)]),
  GameItem(id: 'utility_capacitor', name: 'Capacitor', kind: ItemKind.module, rarity:
Rarity.rare, slot: ModuleSlot.utility, modifiers: [Modifier(StatMod.cooldownPct, -15),
Modifier(StatMod.chargeRateFlat, 4)]),
  GameItem(id: 'utility_overclock', name: 'Overclock Chip', kind: ItemKind.module,
rarity: Rarity.epic, slot: ModuleSlot.utility, modifiers: [Modifier(StatMod.cooldownPct,
-20), Modifier(StatMod.atkPct, 6)]),
  // Consumables (one-shot effects, not equipped)
  GameItem(id: 'use_repair_kit', name: 'Repair Kit', kind: ItemKind.consumable, rarity:
Rarity.common, consumable: ConsumableSpec(ConsumableEffect.healHp, 30)),
  GameItem(id: 'use_energy_cell', name: 'Energy Cell', kind: ItemKind.consumable,
rarity: Rarity.common, consumable: ConsumableSpec(ConsumableEffect.restoreEnergy, 40)),
  GameItem(id: 'use_scrap_cache', name: 'Scrap Cache', kind: ItemKind.consumable,
rarity: Rarity.rare, consumable: ConsumableSpec(ConsumableEffect.grantScrap, 100)),
  GameItem(id: 'use_overcharge_chip', name: 'Overcharge Chip', kind:
ItemKind.consumable, rarity: Rarity.rare, consumable:
ConsumableSpec(ConsumableEffect.fillCharge, 100)),
  // Cosmetics (dog skins, no stats)
  GameItem(id: 'skin_chrome', name: 'Chrome Coat', kind: ItemKind.cosmetic, rarity:
Rarity.rare),
  GameItem(id: 'skin_neon', name: 'Neon Coat', kind: ItemKind.cosmetic, rarity:
Rarity.epic),
  GameItem(id: 'skin_gold', name: 'Gold Coat', kind: ItemKind.cosmetic, rarity:
Rarity.legendary),
];

GameItem catalogById(String id) => kCatalog.firstWhere((it) => it.id == id);
```

Applying modifiers

Sum every equipped module modifier per stat, apply flats first and percentages second, and cap stacked percentages so multiple epics do not snowball.

```
double applyStat(double base, double flatSum, double pctSum) {
  final cappedPct = pctSum.clamp(-60.0, 60.0); // keep stacking sane
  return (base + flatSum) * (1 + cappedPct / 100);
}

CombatLoadout loadoutFromItems(List<GameItem> equipped) {
  double flat(StatMod s) => equipped
      .expand((it) => it.modifiers)
      .where((m) => m.stat == s)
      .fold(0.0, (a, m) => a + m.value);
  // Build your loadout from flat(StatMod.atkFlat), flat(StatMod.defPct), etc.
  // Then in startFight(): finalDef = applyStat(baseDef, flat(StatMod.defFlat),
flat(StatMod.defPct));
```

```
  return CombatLoadout(/* map the sums into your fields */);
}
```

This replaces the single-power loadout from the Combat Overhaul guide (Phase E): read these modifiers instead of one power value.

Notes

- Adapt names: economyProvider (addScrap / spendScrap / addKeys / spendKey), inventoryProvider.add, applyCare returning a changed flag, CareAction, and SharedPreferences vs Isar will differ in your code.
- Persist everything: scrap, keys, inventory, and the starter_granted flag must survive restarts (you already persist combat_level).
- Anti-farm: scrap is granted only on effective care actions; add a short per-action cooldown if needed.
- Premium gems stay earned-only: epic keys cost gems earned from bosses / achievements - no real-money purchases, per your design.
- Consumables / cosmetics: wire Repair Kit / Energy Cell to restore stats, Scrap Cache to grant scrap, Overcharge Chip to fill the combat charge, and cosmetics to swap the dog sprite tint.
- Balance: all prices, weights, and modifier values are starting points - tune after playing.